

TP3 Docker - Persistance de données

Claire CAUSSE

Os : MacOS

Partie 1 – Echanges de fichiers

1/

Commande : `docker run -d --name www -p 8080:80 nginx:latest`

```
tpintegrationcontinue-clairecausse on 12 dev [?] is v1.0-SNAPSHOT via  took 10s
0 9% ) docker run -d --name www -p 8080:80 nginx:latest
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
51365f04b688: Pull complete
d9b6d209211f: Pull complete
184706839d41: Pull complete
59679015de73: Pull complete
7d3a535ffc2f: Pull complete
0469ebd60c79: Pull complete
3db9f1af9e10: Pull complete
Digest: sha256:1beed3ca46acebe9d3fb62e9067f03d05d5bfa97a00f30938a0a3580563272ad
Status: Downloaded newer image for nginx:latest
570b8b0781946814420aad841ebcff08017f6689e617a4ef59ea893c4b5c8b19
(base)
```

Le conteneur est créé et exécute le serveur web nginx, avec le port 80 du conteneur redirigé vers le port 8080 de la VM.

Commande : `docker ps`

```
tpintegrationcontinue-clairecausse on 12 dev [?] is v1.0-SNAPSHOT via  took 3s
0 9% ) docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
570b8b078194   nginx:latest "/docker-entrypoint..." 28 seconds ago Up 28 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp www
0fe33381722e   ubuntu    "/bin/bash"              10 minutes ago Up 9 minutes   -                               conteneur
(base)
```

Elle permet de constater la redirection de ports affichée dans la colonne *PORTS* sous la forme `0.0.0.0:8080->80/tcp`.

2/

http://localhost:8080

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

En entrant cette adresse dans le navigateur de la machine hôte, la page d'accueil par défaut de nginx s'affiche, confirmant que la redirection de ports fonctionne correctement.

3/

Commande : `mkdir -p ~/SiteWeb && echo "Bienvenue sur le serveur web du conteneur." > ~/SiteWeb/index.html`

L localhost:8080

Bienvenue sur le serveur web du conteneur.

Le dossier *SiteWeb* est créé (s'il n'existe pas déjà) et le fichier *index.html* y est ajouté avec le texte indiqué.

4/

Commande : `docker cp ~/SiteWeb/index.html www:/usr/share/nginx/html/index.html`

```
tpintegrationcontinue-clairecausse on ? dev [?+] is v1.0-SNAPSHOT via
• > docker cp ~/SiteWeb/index.html www:/usr/share/nginx/html/index.html
Successfully copied 2.05kB to www:/usr/share/nginx/html/index.html
(base)
tpintegrationcontinue-clairecausse on ? dev [?+] is v1.0-SNAPSHOT via
```

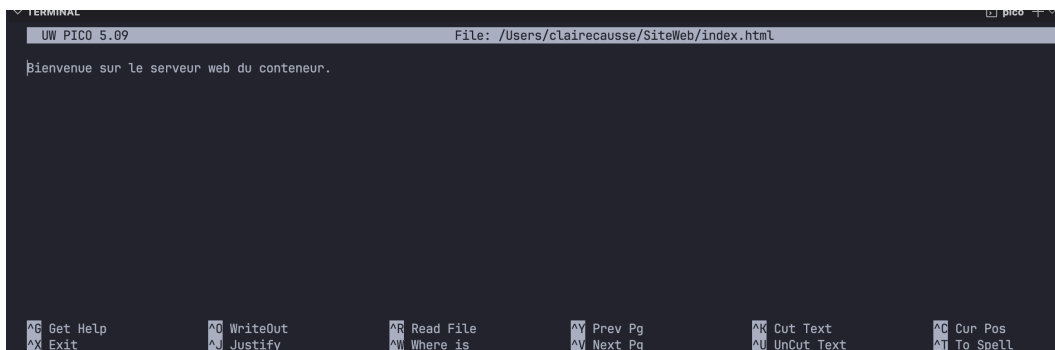
Cette commande copie ton fichier `index.html` local dans le dossier `/usr/share/nginx/html` du conteneur `www`.

Ensuite, ouvre ton navigateur sur **http://localhost:8080** :

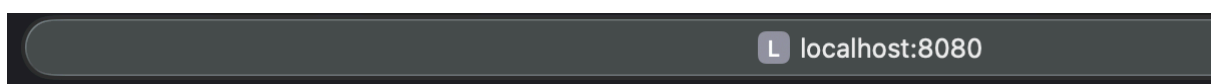
la page affichera **"Bienvenue sur le serveur web du conteneur."** à la place de la page d'accueil par défaut de Nginx.

5/

Commande : `nano ~/SiteWeb/index.html`



On peut faire ainsi une petite modification dans le fichier.



Bienvenue sur le serveur web du conteneur, voici la page d'accueil du serveur de la VM.

6/

Commande pour consulter le fichier dans le conteneur :

```
docker exec -it www cat /etc/nginx/conf.d/default.conf
```

Commande pour le copier sur la VM :

```
docker cp www:/etc/nginx/conf.d/default.conf ~/default.conf
```

```
tpintegrationcontinue-clairecausse on 1? dev [?+] is v1.0-SNAPSHOT via
> docker cp www:/etc/nginx/conf.d/default.conf ~/default.conf
Successfully copied 3.07kB to /Users/clairecausse/default.conf
(base)
tpintegrationcontinue-clairecausse on 1? dev [?+] is v1.0-SNAPSHOT via
```

Pour le consulter localement avec :

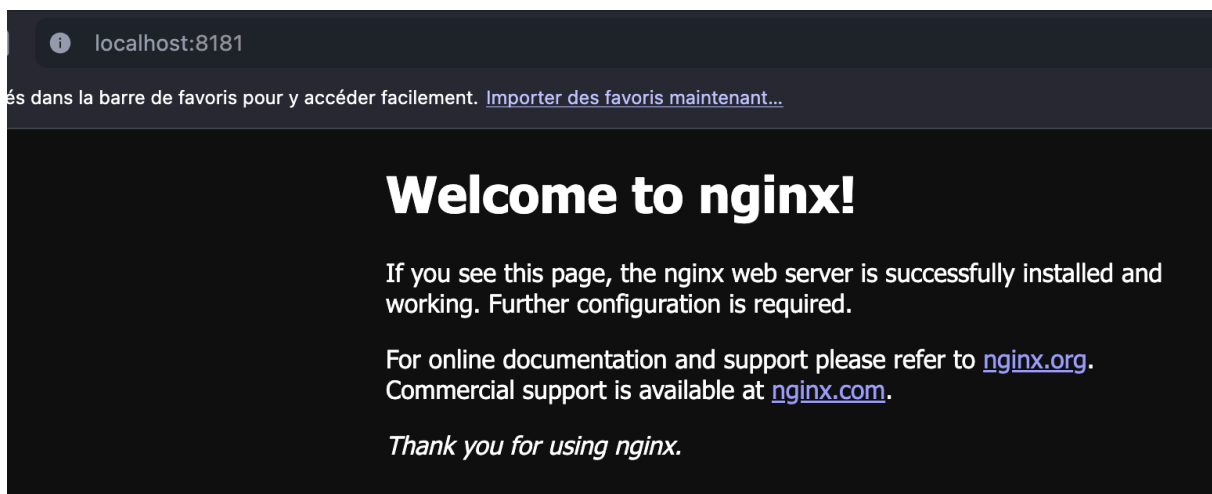
```
cat ~/default.conf
```

7/

Commande : `docker run -d --name www2 -p 8181:80 nginx`

```
tpintegrationcontinue-clairecausse on 1? dev [?+] is v1.0-SNAPSHOT via
> docker run -d --name www2 -p 8181:80 nginx
bc131b2a3ad889029d2e13b1a7aa3dabdc0502796abd83e9c5ff84379fd8840e
(base)
tpintegrationcontinue-clairecausse on 1? dev [?+] is v1.0-SNAPSHOT via
```

Le conteneur www2 démarre un second serveur web nginx, cette fois relié au port 8181 de la VM.



Dans ton navigateur, ouvre **http://localhost:8181** :

la page d'accueil par défaut de nginx s'affiche, confirmant que la redirection de port fonctionne pour ce second conteneur.

8/

Commande :

```
docker cp www:/usr/share/nginx/html/index.html .
```

```
docker cp index.html www2:/usr/share/nginx/html/
```

```
tpintegrationcontinue-clairecausse on ? dev [?] is v1.0-SNAPSHOT via
) docker cp www:/usr/share/nginx/html/index.html .
Successfully copied 2.05kB to /Users/clairecausse/MacBook de Claire/IUT/M1/M1/Principes et outils pour DevOps/tpintegrationcontinue-clairecausse
(base)
tpintegrationcontinue-clairecausse on ? dev [?] is v1.0-SNAPSHOT via
) docker cp index.html www2:/usr/share/nginx/html/
Successfully copied 2.05kB to www2:/usr/share/nginx/html/
(base)
tpintegrationcontinue-clairecausse on ? dev [?] is v1.0-SNAPSHOT via
```

Cette combinaison copie le fichier `index.html` du conteneur **www** vers **www2** sans passer par un fichier temporaire sur la VM.

En rechargeant <http://localhost:8181>, tu verras le même contenu que sur <http://localhost:8080>.

9/

Commande :

```
docker cp www:/usr/share/nginx/html/index.html - | tar -xO | docker cp - www2:/usr/share/nginx/html/
```

```
tpintegrationcontinue-clairecausse on ? dev [?] is v1.0-SNAPSHOT via
) docker exec www tar -cf - -C /usr/share/nginx/html index.html | docker exec -i www2 tar -xf - -C /usr/share/nginx/html
(base)
tpintegrationcontinue-clairecausse on ? dev [?] is v1.0-SNAPSHOT via
```

Cette combinaison utilise un flux Unix :

- `docker cp ... -` extrait le fichier du conteneur **www** vers la sortie standard,
- `tar -xO` le décompresse et envoie son contenu,
- `docker cp - ...` l'importe dans le dossier du conteneur **www2**.



Bienvenue sur le serveur web du conteneur, voici la page d'accueil du serveur de la VM.

Ainsi, le fichier *index.html* est copié directement d'un conteneur à l'autre, sans passer par la VM.

10/

Commande : `docker rm -f www www2`

Elle arrête et supprime simultanément les deux conteneurs nginx créés lors de cette partie.

Partie 2 – Volumes mappés (bind)

1/

Commande : `cd ~/SiteWeb`

Commande : `echo "server { listen 80; root /usr/share/nginx/html; }" > default.conf`

```
(base)
tpintegrationcontinue-clairecausse on ? dev [?] is v1.0-SNAPSHOT via
> cd ~/SiteWeb
(base)
~/SiteWeb
> echo "server { listen 80; root /usr/share/nginx/html; }" > default.conf
(base)
~/SiteWeb
```

Si le fichier n'existe pas, il est créé. Le répertoire contient alors `index.html` et `default.conf`.

2/

Commande : `docker run -d --name www -p 8080:80 -v ~/SiteWeb/index.html:/usr/share/nginx/html/index.html nginx`

```
~/SiteWeb
> docker run -d --name www -p 8080:80 -v ~/SiteWeb/index.html:/usr/share/nginx/html/index.html nginx
e21ac937e0a3a0fac9f861374fb5f1fdfa5af5e1266aebc35c8fa2007d09ea92
(base)
```

Le fichier `index.html` de la VM est directement utilisé par nginx dans le conteneur. En accédant à `http://localhost:8080`, toute modification locale du fichier est visible immédiatement.



Partie 2 Volumes mappés (bind)

3/

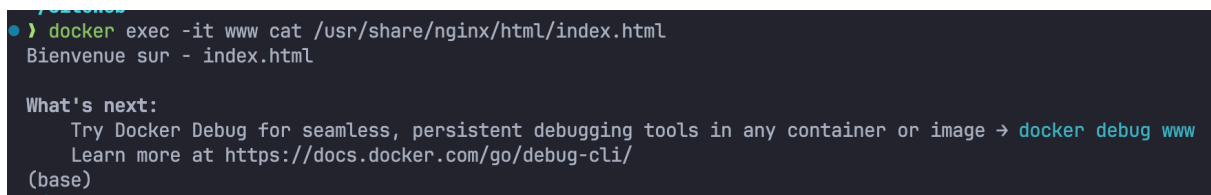
Commande : `echo "Bienvenue sur le serveur WEB du conteneur." > ~/SiteWeb/index.html`

La modification est instantanément reflétée dans le conteneur grâce au montage du fichier. En rechargeant `http://localhost:8080`, la nouvelle version du texte s'affiche.



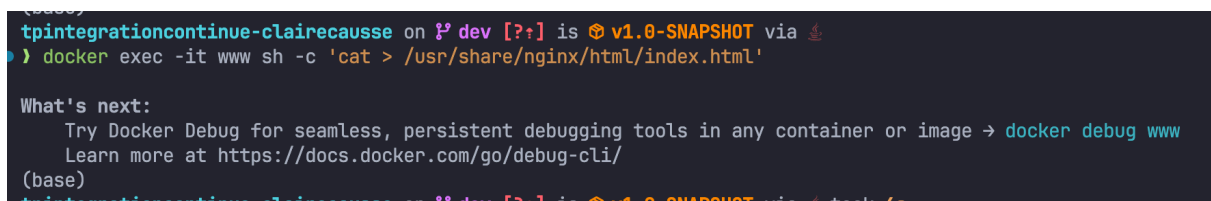
4/

Commande : `docker exec -it www cat /usr/share/nginx/html/index.html`



5/

Commande : `docker exec -it www sh -c 'cat > /usr/share/nginx/html/index.html'`



Le nouveau contenu remplace directement l'ancien. Après **Ctrl + D**, la page `http://localhost:8080` affiche la version modifiée, car le fichier est lié avec celui de la VM.

6/

Commande : `cat ~/SiteWeb/index.html`



```
tpintegrationcontinue-clairecausse
● > cat ~/SiteWeb/index.html
(base)
```

Elle affiche le contenu du fichier sur la VM (ici vide du à la dernière commande), identique à celui modifié dans le conteneur grâce au lien entre les deux fichiers.

7/

Commande : `docker rm -f www`

Elle arrête et supprime définitivement le conteneur.

8/

Commande : `docker run -d --name www -p 8080:80 -v ~/SiteWeb:/usr/share/nginx/html nginx`



```
tpintegrationcontinue-clairecausse on 1? dev [?] is v1.0-SNAPSHOT via
● > docker run -d --name www -p 8080:80 -v ~/SiteWeb:/usr/share/nginx/html nginx
9533446e5dadfca92dc5b42567b26ed921d7025d9803df84a163bdd51c3e2f5d
(base)
```

Le répertoire *SiteWeb* de la VM est monté dans le dossier du serveur web. En rechargeant `http://localhost:8080`, la page s'affiche directement à partir des fichiers du répertoire local donc notre ancien fichier index.html.

L localhost:8080

9/

Commande : `echo "Bienvenue sur le NOUVEAU serveur web du conteneur." > ~/SiteWeb/index.html`

L localhost:8080

Bienvenue sur le NOUVEAU serveur web du conteneur.

En rechargeant `http://localhost:8080`, la modification est immédiatement visible, car le dossier *SiteWeb* est monté directement dans le conteneur.

10/

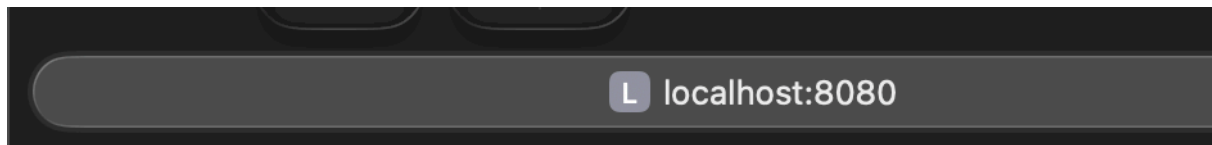
Commande : `docker exec -it www sh -c 'cat > /usr/share/nginx/html/index.html'`

```
tpintegrationcontinue-clairecausse on P dev [?+] is v1.0-SNAPSHOT via
) docker exec -it www sh -c 'cat > /usr/share/nginx/html/index.html'

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug www
  Learn more at https://docs.docker.com/go/debug-cli/
(base)
tpintegrationcontinue-clairecausse on P dev [?+] is v1.0-SNAPSHOT via took 16s
```

On modifie le contenu, puis on appuie sur **Ctrl + D** pour enregistrer.

La nouvelle version est visible immédiatement sur `http://localhost:8080` et dans le fichier `~/SiteWeb/index.html` de la VM, puisque le répertoire est monté dans le conteneur.

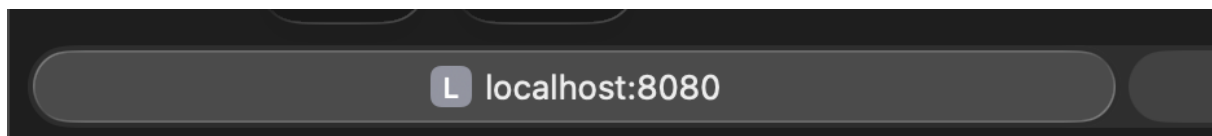


11/

Commande : `echo "Bienvenue sur le serveur WEB du conteneur (modifié localement)." > ~/SiteWeb/index.html`

Le fichier étant monté dans le conteneur, la modification est immédiatement visible sur <http://localhost:8080>.

```
(base)
tpintegrationcontinue-clairecausse on ? dev [?] is v1.0-SNAPSHOT via
● ) echo "Bienvenue sur le serveur WEB du conteneur (modifié localement)." > ~/SiteWeb/index.html
(base)
tpintegrationcontinue-clairecausse on ? dev [?] is v1.0-SNAPSHOT via
```



Bienvenue sur le serveur WEB du conteneur (modifié localement).

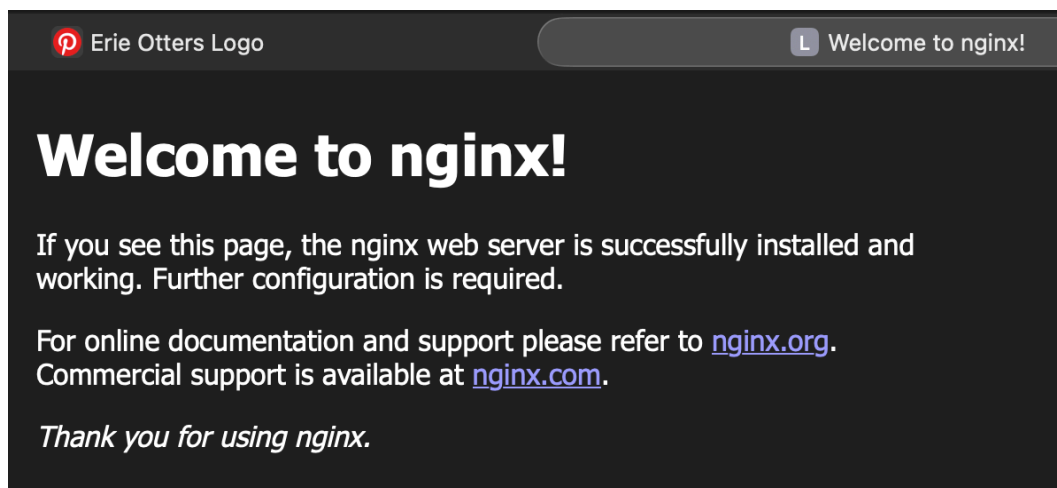
12/

```
tpintegrationcontinue-cla
● ) docker rm -f www
www
(base)
tpintegrationcontinue-cla
```

13/

Commande : `docker run -d --name www -p 8080:80 nginx`

```
tpintegrationcontinue-clairecausse on ? dev [?!] is v1.0-SNAPSHOT
● > docker run -d --name www -p 8080:80 nginx
93c3f9733e0af646d958378f1a79b83d5ba5248a30743d9427a4e70413030948
(base)
```



Le conteneur utilise uniquement le fichier `index.html` par défaut inclus dans l'image nginx.

Dans le navigateur, <http://localhost:8080> affiche la page d'accueil **par défaut de Nginx** ("Welcome to nginx!").

14/

```
tpintegrationcontinue-cla
● > docker rm -f www
www
(base)
tpintegrationcontinue-cla
```

15/

Commande : `docker run -d --name www -p 8080:80 -v ~/SiteWebInexistant:/usr/share/nginx/html nginx`

Comme le dossier `~/SiteWebInexistant` n'existe pas, Docker crée un répertoire vide.

Dans le navigateur, <http://localhost:8080> affiche une erreur **403 Forbidden**, car **nginx** ne trouve aucun fichier **index.html** à servir.

```
(base)
tpintegrationcontinue-clairecausse on 1 dev [?] is v1.0-SNAPSHOT via
• > docker run -d --name www -p 8080:80 -v ~/SiteWebInexistant:/usr/share/nginx/html nginx
59e1d226c9aefa107ff3cd9162560b1f9ab2fa0bd6181793d633f33ce02a11eb
(base)
tpintegrationcontinue-clairecausse on 1 dev [?] is v1.0-SNAPSHOT via
```



403 Forbidden

nginx/1.29.3

Partie 3 – Volumes gérés par Docker

1/

Commande : `docker volume create site`

```
tpintegrationcontinue-clairecausse on 1 dev [?] is v1.0-SNAPSHOT via
> docker volume create site
site
(base)
```

Docker crée un **volume nommé** `site` dans l'espace de stockage des volumes (`/var/lib/docker/volumes/`).

Ce volume est vide par défaut et pourra ensuite être monté dans un conteneur pour **stocker des données de manière persistante**, indépendamment de la suppression du conteneur.

2/

Commande : `docker volume ls`

```
tpintegrationcontinue-clairecausse on ? dev [?] is v1.0-SNAPSHOT via
> docker volume ls
DRIVER      VOLUME NAME
local       0ceac4efe5adbf02db498fe05eae9210f6390c7d694deda72facd61b932546b1
local       29fdd67e7f94e16459ba5770196fe220442842c442b67374197342af2f693878
local       98df11c97d8cc5166f6bf53183e080dd2ece470dd9f41c5535ee64c0b89c9cef
local       ae2i-backup_mongo_data
local       d4c08a2f2d9b52e5eefc4e4f01a226f385c7fd469d5cee3a51b50d4c9714fb00
local       d5de112b67638362dd5e95c5c63b94ab0998d1f786161924586a3e2f146892ea
local       pharmacymis_api_PMISSDBData
local       s5a01-ae2i_mongo_data
local       sae-s5a01-2024-sujet03_mongo_data
local       site
(base)
```

Dans le résultat, on retrouve une table avec les colonnes **DRIVER** (souvent `local`) et **VOLUME NAME** — par exemple, on y verra apparaître le volume `site` créé précédemment.

3/

Commande : `docker run -d --name www -p 8080:80 -v site:/usr/share/nginx/html nginx`

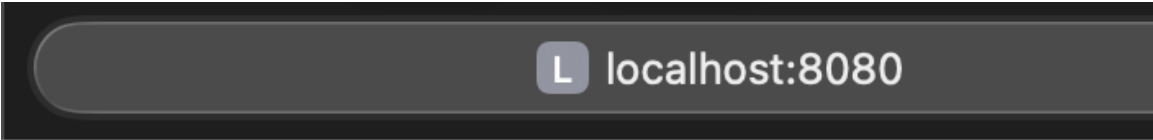
```
(base)
tpintegrationcontinue-clairecausse on ? dev [?] is v1.0-SNAPSHOT via
> docker run -d --name www -p 8080:80 -v site:/usr/share/nginx/html nginx
6ecf5db3ed7fb8f7da622bed47f601fd1731acd1e4d45fd5ab8db50a35550594
(base)
```

Le conteneur `www` est créé à partir de l'image `nginx`. Son dossier `/usr/share/nginx/html` est monté sur le volume `site`, ce qui le rend persistant.

Dans le navigateur, <http://localhost:8080> affiche une erreur 403 Forbidden, car le volume est vide et `nginx` ne trouve aucun fichier `index.html` à servir.

4/

Une fois dans le conteneur, commande `echo "Bonjour depuis le conteneur" > /usr/share/nginx/html/index.html`

A dark-themed terminal window with a rounded header bar. The header bar contains a small square icon with the letter 'L' on the left and the text 'localhost:8080' on the right.

Bonjour depuis le conteneur

Dans le navigateur, <http://localhost:8080> affiche désormais la phrase ajoutée, preuve que le fichier est bien pris en compte par nginx.

5/

Commande:

```
exit
```

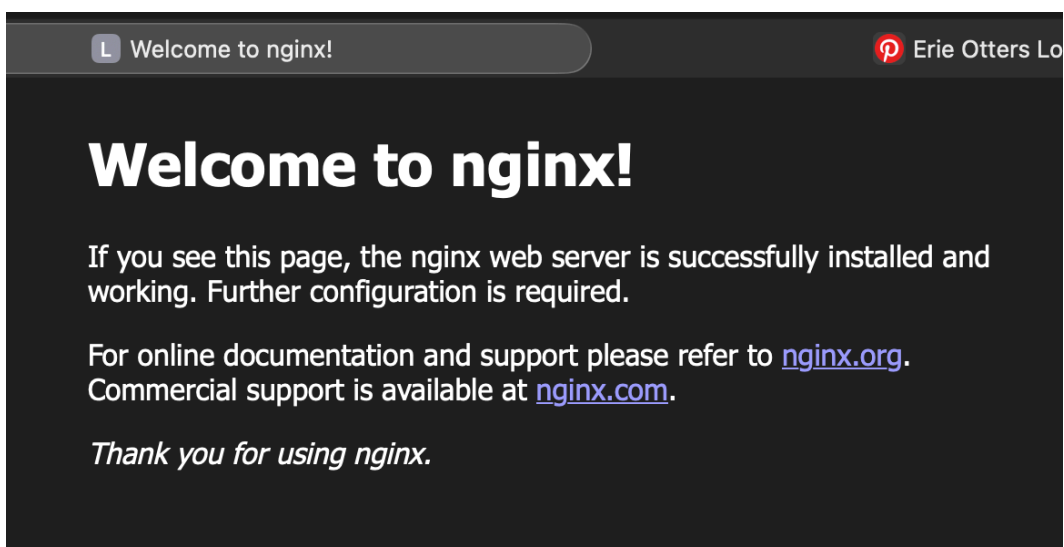
```
docker rn -f www
```

6/

Commande : `docker run -d --name www -p 8080:80 nginx`

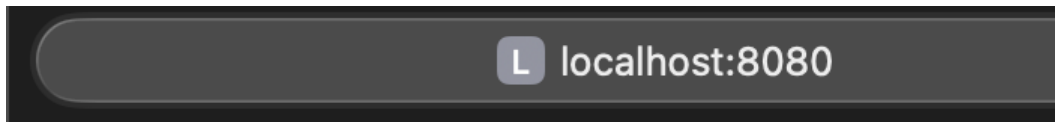
Le conteneur www est recréé sans montage du volume site. Il utilise donc les fichiers par défaut de l'image nginx.

Dans le navigateur, <http://localhost:8080> affiche la page d'accueil par défaut de nginx, indiquant que le serveur fonctionne et sert désormais les fichiers internes à l'image.



7/

Commande : `docker run -d --name www -p 8080:80 -v site:/usr/share/nginx/html nginx`



Bonjour depuis le conteneur

Le conteneur **www** est recréé avec le volume **site** monté dans le dossier `/usr/share/nginx/html`.

Dans le navigateur, <http://localhost:8080> affiche le contenu personnalisé précédemment enregistré dans le fichier index.html, preuve que le volume a conservé les données même après la suppression de l'ancien conteneur.

8/

Commande : `docker volume ls -f dangling=true`

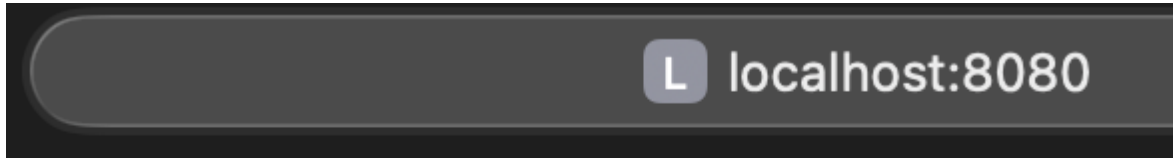
```
tpintegrationcontinue-clairecausse on P dev [?+] is v1.0-SNAPSHOT via
● ) docker volume ls -f dangling=true
DRIVER      VOLUME NAME
local       0ceac4efe5adb02db498fe05eae9210f6390c7d694deda72facd61b932546b1
local       29fdd67e7f94e16459ba5770196fe220442842c442b67374197342af2f693878
local       98df11c97d8cc5166f6bf53183e080dd2ece470dd9f41c5535ee64c0b89c9cef
local       ae2i-backup_mongo_data
local       d4c08a2f2d9b52e5eefc4e4f01a226f385c7fd469d5cee3a51b50d4c9714fb00
local       d5de112b67638362dd5e95c5c63b94ab0998d1f786161924586a3e2f146892ea
local       pharmacymis_api_PMISDBData
local       s5a01-ae2i_mongo_data
local       sae-s5a01-2024-sujet03_mongo_data
local       site
(base)
```

Cette commande filtre et affiche uniquement les volumes orphelins, c'est-à-dire ceux non utilisés par aucun conteneur.

Le résultat liste les volumes disponibles avec le driver `local` et indique les volumes dangling, qui peuvent être supprimés pour libérer de l'espace.

9/

Commande : `docker run -d --name www -p 8080:80 -v site:/usr/share/nginx/html:ro nginx`



Bonjour depuis le conteneur

Le conteneur `www` est recréé en montant le volume `site` en lecture seule (`:ro`).

Dans le navigateur, <http://localhost:8080> affiche toujours la page personnalisée du volume, car les fichiers sont accessibles en lecture.

Cependant, toute tentative de modification du fichier `/usr/share/nginx/html/index.html` dans le conteneur échoue avec une erreur « Read-only file system », car le volume est monté en lecture seule.

10/

Commande : `docker volume rm site`

Docker tente de supprimer le volume nommé `site`.

```
(base)
tpintegrationcontinue-clairecausse on ? dev [?] is v1.0-SNAPSHOT via
❯ docker volume rm site
Error response from daemon: remove site: volume is in use - [ea42713bea3bc4c2f68a958beca77ea36c707d59fe0bdbb23a113ec28ab308c0]
❯ (base)
```

L'opération échoue si le volume est encore utilisé par un conteneur actif, et affiche alors un message d'erreur indiquant que le volume est en cours d'utilisation. Il faut d'abord supprimer ou détacher le conteneur avant de pouvoir effacer le volume.

11/

Commande : `docker volume inspect site`

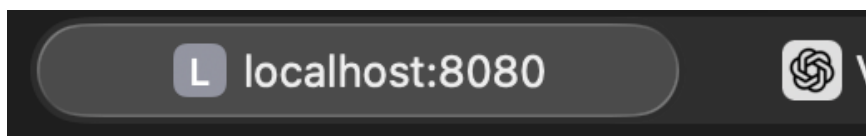

```
(base)
tpintegrationcontinue-clairecausse on 8% dev [?] is v1.0-SNAPSHOT via
8% > docker volume inspect site
[
  {
    "CreatedAt": "2025-11-05T14:22:31Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/site/_data",
    "Name": "site",
    "Options": null,
    "Scope": "local"
  }
]
(base)
```

Cette commande affiche les informations détaillées du volume site, notamment le chemin d'accès réel sur la machine hôte dans le champ `"Mountpoint"`.

En se rendant dans ce répertoire, par exemple avec `cat` `/var/lib/docker/volumes/site/_data/index.html`, on peut consulter le contenu du fichier `index.html` directement depuis l'hôte, confirmant que les données sont bien stockées de manière persistante dans le volume.

12/

Commande : `echo "Modification depuis l'hôte" > /var/lib/docker/volumes/site/_data/index.html`



Modification depuis l'hôte

Le fichier `index.html` du volume site est modifié directement depuis l'hôte, dans le répertoire `/var/lib/docker/volumes/site/_data`.

Dans le navigateur, <http://localhost:8080> affiche immédiatement le nouveau texte, montrant que nginx sert bien le contenu actualisé du volume.

13/

Commande : `docker volume rm site`

Docker supprime le volume nommé site.

Le volume est encore utilisé par un conteneur, la commande échoue avec un message d'erreur. Il faut alors supprimer le conteneur associé avec `docker rm -f www` avant de relancer la suppression du volume. Une fois le volume supprimé, les données qu'il contenait sont définitivement effacées.

14/

Commande : `docker run -d --name www -p 8080:80 -v /usr/share/nginx/html nginx`

Docker crée un conteneur www avec un volume anonyme, c'est-à-dire un volume sans nom attribué explicitement.

```
tpintegrationcontinue-clairecausse on ♀ dev [?] is 📦 v1.0-SNAPSHOT via 🐘
● ) docker volume ls
DRIVER      VOLUME NAME
local       0ceac4efe5adbf02db498fe05eae9210f6390c7d694deda72facd61b932546b1
local       3daf91658e1322f4d8354d7b6aafa862a652d11884b246b907802ae17bd071b4
local       29fdd67e7f94e16459ba5770196fe220442842c442b67374197342af2f693878
local       98df11c97d8cc5166f6bf53183e080dd2ece470dd9f41c5535ee64c0b89c9cef
local       ae2i-backup_mongo_data
local       d4c08a2f2d9b52e5eefc4e4f01a226f385c7fd469d5cee3a51b50d4c9714fb00
local       d5de112b67638362dd5e95c5c63b94ab0998d1f786161924586a3e2f146892ea
local       pharmacymis_api_PMISSDBData
local       s5a01-ae2i_mongo_data
local       sae-s5a01-2024-sujet03_mongo_data
(base)
```

Dans la liste obtenue avec `docker volume ls`, un nouveau volume apparaît avec un nom généré automatiquement (une suite de caractères aléatoires).

15/

Commande : `docker inspect www`

Cette commande affiche tous les détails du conteneur www.

```

    },
    "Mounts": [
      {
        "Type": "volume",
        "Name": "3daf91658e1322f4d8354d7b6aafa862a652d11884b246b907802ae17bd071b4",
        "Source": "/var/lib/docker/volumes/3daf91658e1322f4d8354d7b6aafa862a652d11884b246b907802ae17bd071b4/_data",
        "Destination": "/usr/share/nginx/html",
        "Driver": "local",
        "Mode": "",
        "RW": true,
        "Propagation": ""
      }
    ]
  },
  ]
}

```

En cherchant la section `"Mounts"`, on peut vérifier qu'un volume a bien été créé et monté.

On y trouve notamment :

- `"Type": "volume"` indiquant qu'il s'agit d'un volume Docker
- `"Name"` correspondant au nom (ou identifiant) du volume
- `"Destination"` montrant le chemin dans le conteneur où il est monté

Cela confirme qu'un volume est bien associé à ce conteneur.

16/

Commande : `docker rm -v www`

Cette commande supprime le conteneur `www` **et** le volume anonyme associé en même temps.

L'option `-v` indique à Docker de **supprimer automatiquement les volumes** attachés à ce conteneur, même si leur identifiant n'est pas connu.

Ainsi, le volume anonyme est supprimé sans avoir besoin de le nommer ou de le retrouver manuellement.